FORMATO DE DESARROLLO E INTEGRACIONES

FT-SL-011-APIs
Botón Ecommerce Credi10

STF GROUP S.A.
INFORMATICA – Py. CREDI10 SOFTWARE PROPIO

Cali, jueves 19 de febrero del 2024

TABLA DE CONTENIDO

INTRODUCCION 3
1. DETALLE TECNICO 4
1.1 Detallar componentes existentes..... 4
1.2 Detallar funcionalidades 5
1.3 Observaciones y/o Comentarios..... 18
2. CASOS DE PRUEBAS UNITARIAS 19

INTRODUCCION

Este documento muestra en detalle todas las actividades de la solución a desarrollar con sus dependencias, entradas y salidas. Así como los casos de pruebas unitarias realizadas a la solución.

NOTA: El desarrollo de cualquier solución de requerimiento debe seguir los estándares de desarrollo que define Arquitectura.

1. DETALLE TECNICO

1.1 Descripción del Requerimiento:

1.1.1 Requerimiento

Disponer API para suministrar información a paginas ecommerce del cupo que tienen los clientes con credi10 y poder realizar la compra por este medio de pago.

Se han establecido múltiples requerimientos donde cada uno presenta funcionalidades y procesos específicos que permitan la generación de información para link de recaudos.

A continuación, se detalla los procesos importantes para cumplir con el requerimiento.

- Crear parámetros de configuración para las APIs de Credi10.
- Crear endpoint para consultar información del cliente, valide si tiene cupo con credi10, genere OTP de compra y lo envié por correo y SMS para seguir con el proceso de compra desde el ecommerce.
- Crear endpoint para crear el pagare de la compra realizada desde el ecommerce.

1.2 Detallar componentes existentes

Para el desarrollo de los requerimientos se cuenta con 1 solución desarrollada en .Net Framework 4.8 llamada “Créditos”.

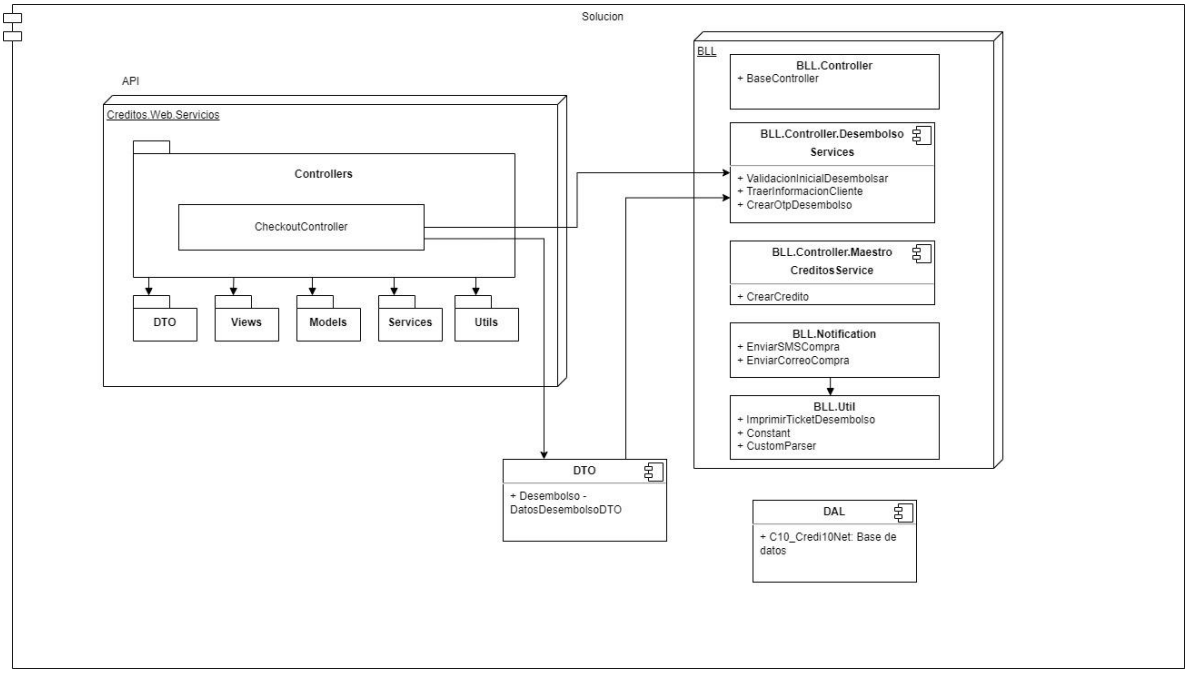
Se realiza creación del proyecto web API “*Creditos.Web.Servicios*” donde se establece el funcionamiento de los requerimientos correspondientes a Link de recaudo, botón ecommerce e integración B2C.

Nombre del Componente	Descripción
SOLUCION	
Creditos.Web.Servicios	API CheckoutController
Creditos.Web.Admin	Controladores y vistas implementadas para cumplir con el propósito de informar el valor del pago y recibir el valor que quieran aplicar.
Creditos.BLL	Proyecto que contiene las clases y métodos que implementan la lógica de negocio de la aplicación, esta biblioteca de clases será el encargado de calcular los pagos, procesarlos, generar y enviar el ticket. Este proyecto se integra con el proyecto Web.Admin, el proyecto DAL y el proyecto DTO.

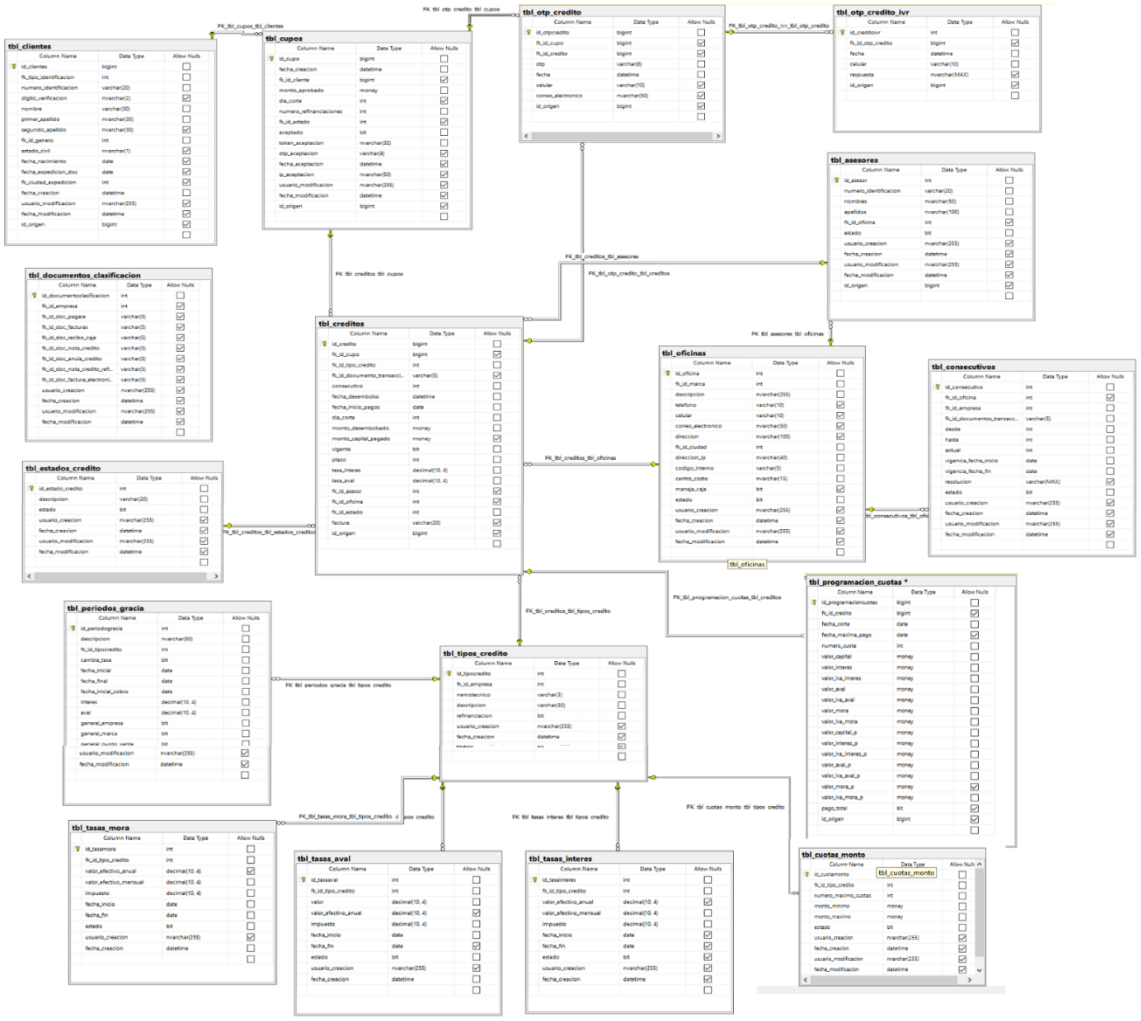
Creditos.DAL	Proyecto encargado de tener el acceso y la interacción a la base de datos C10_Credi10NET de la solución.
Creditos.DTO	Proyecto que contiene los diferentes modelos usados en toda la aplicación.
Rotativa	Herramienta de línea de comandos para representar HTML en PDF, encargado de crear los tickets de impresión del pago.
BASE DE DATOS	
C10_Credi10NET	Tablas y Procedimientos almacenados

1.3 Detallar funcionalidades

1.3.1 Diagrama de Componentes



1.3.2 MER



Parametrizaciones

Descripción	Grupo	Valor
Link conexión consumo API pagos	Link	https://apicredinet.studiof.com.co:xxx/payment/paymentLink

1.3.3 Proceso

Se implementa el componente CheckoutController encargado de recibir por medio de la clase Ecommerce una petición encriptada que tendrá 2 tipos de operaciones. Para este en

el objeto encriptado se establece el valor “consultar” y consiste en realizar la búsqueda de información del cliente suministrado y obtener la información de cupos aprobados,

disponibles e identificar si tiene capacidad, generando OTP y posterior envío vía correo electrónico y SMS para continuar con el proceso.

Para esto se necesita recibir un objeto JSON en el cuerpo de la solicitud POST como el que se observa a continuación:


```
{"datos":"CoIqISGDjiQV5NB\/CPmMb+9JUwlz\/4Ygt9eF+9MwoVoRkOk
    +qixQKDwEfnujHO0Zt9FjU2thwI71z0Gu6rOg3urmhFovY7jN9rl+lo+WvUPGjuhqB4wd3lQQSKsJZATXz0oBgsKn3A\/
    9hTnsPEtC1b2VwXkuMSsSFLKt7TZpoVNI6Q4B1YSnzyd94RiO6bbNXZm3FlRBhZ4pXm0exEBAoB58rXA3ud3mOSfdPK+yzPEO1
    +M\/5Lvfj+HY44fCiVTuM2IZ+L6y9Lnp6VQsh026n4GwxQGo0ykQbZIYT6fYKtGiVENnAqyTaETMfXbwxyayIK76YlmPm\/
    CAMXe6Jz++YlGys7\/xFvi4m7BIs0SNlwE="}
```


Donde el contenido del atributo “datos” corresponde a una encriptación del siguiente objeto


```
{
“operacion”: “consultar”,
“identificacion”: “16736984”,
“celular”: “3137538636”,
“correo”: correo@gmail.com,
“monto”: “50000”,
“plazo”:”1”,
“usuario”: “xxx”,
“password”: “xxx”
}
```



```
public class CheckoutController : Controller
{
    0 referencias
    public async Task<ActionResult> Ecommerce()
    {
        DesembolsosServices desembolso = new DesembolsosServices();
        MaestrosCreditosService maestrosCreditosService = new MaestrosCreditosService();
        EncryptionAES encryptionAES = new EncryptionAES();
        OperationResponse result = new OperationResponse();
        UserServices userServices = new UserServices();
        PreaprobadoController Preaprobado = new PreaprobadoController();
        var usuario = "ApiEcommerce";
        object dataObj = new object();
        object response = new object();

        try
        {
            PermitDTO permit = new PermitDTO();
            permit.Crear = true;

            // Desencriptar
            Stream req = Request.InputStream;
            req.Seek(0, SeekOrigin.Begin);
            string jsonRequest = new StreamReader(req).ReadToEnd();

            CheckoutDto checkoutDto = JsonConvert.DeserializeObject<CheckoutDto>(jsonRequest);
            string valueEncrypted = JsonConvert.SerializeObject(checkoutDto.datos);
            var decryptedData = encryptionAES.Decrypt(valueEncrypted.Replace("\"", ""));

            DataCheckoutDto dataCheckoutDto = JsonConvert.DeserializeObject<DataCheckoutDto>(decryptedData);
            usuario = dataCheckoutDto.Usuario;
            bool userChecked = userServices.ValidateUser(dataCheckoutDto.Usuario, dataCheckoutDto.Password);
            if (!userChecked)
            {
                dataObj = new
                {
                    status = "error",
                    data = new object(),
                    code = 405,
                    mensaje = "No autorizado"
                };
            }
            else
            {

                if (dataCheckoutDto.Operacion == "consultar")...
                else if (dataCheckoutDto.Operacion == "compensar")...
                else
                {
                    dataObj = new
                    {
                        status = "error",
                        data = new List<string>(),
                        code = 405,
                        mensaje = "Opción no disponible"
                    };
                }

            }
        }
        catch (Exception ex)
        {
            dataObj = new
            {
                status = "error",
                data = new List<string>(),
                code = 405,
                mensaje = ex.Message
            };
        }

        var encryptedData = encryptionAES.Encrypt(JsonConvert.SerializeObject(dataObj));
        response = new
        {
            datos = encryptedData
        };

        return Json(response);
    }
}
```

Al realizar cualquier solicitud a la API de CheckoutController entrará al método asíncrono llamado Ecommerce que será el encargado de procesar todas las solicitudes y dependiendo de la operación recibida en el cuerpo de la petición hacer una u otra tarea.

```
PermitDTO permit = new PermitDTO();
permit.Crear = true;

// Desencriptar
Stream req = Request.InputStream;
req.Seek(0, SeekOrigin.Begin);
string jsonRequest = new StreamReader(req).ReadToEnd();

CheckoutDto checkoutDto = JsonConvert.DeserializeObject<CheckoutDto>(jsonRequest);
string valueEncrypted = JsonConvert.SerializeObject(checkoutDto.datos);
var decryptedData = encryptionAES.Decrypt(valueEncrypted.Replace("\"", ""));

DataCheckoutDto dataCheckoutDto = JsonConvert.DeserializeObject<DataCheckoutDto>(decryptedData);
usuario = dataCheckoutDto.Usuario;
bool userChecked = userServices.ValidateUser(dataCheckoutDto.Usuario, dataCheckoutDto.Password);
if (!userChecked)
{
    dataObj = new
    {
        status = "error",
        data = new object(),
        code = 405,
        mensaje = "No autorizado"
    };
}
else...
```

El objeto json recibido en la petición se desencripta con claves específicas y se inicia con el proceso validación del usuario y password suministrados en el objeto, si no son correctas

las credenciales el sistema retornará “No Autorizado”, en caso de ser correctas, se obtendrá la información necesaria para devolver el objeto especificado en la API.



```
if (dataCheckoutDto.Operacion == "consultar")
{
    long? quotaId = null;
    decimal? amountApproved = null;
    decimal? amountAvailable = null;
    decimal? balance = null;

    using (var dbContext = new BaseContext())
    {
        var client = dbContext.tbl_clientes.FirstOrDefault(x => x.numero_identificacion == dataCheckoutDto.Identificacion);

        // TODO: Acomodar alguna vez la logica
        if (client == null) {
            var outputData = new
            {
                idcupo = 0,
                identificacion = dataCheckoutDto.Identificacion,
                diacorte = 0,
                totalcupo = 0,
                saldo = 0,
                verificacorreo = "No",
                verificacelular = "No",
                verificacapacidad = "No",
                capacidad = 0,
                token = String.Empty,
                id_vtex = String.Empty,
                comentario = "No tiene Cupo"
            };

            dataObj = new
            {
                status = "error",
                data = outputData,
                code = 405,
                mensaje = result.Message
            };

            var encryptedData2 = encryptionAES.Encrypt(JsonConvert.SerializeObject(dataObj));
            response = new
            {
                datos = encryptedData2
            };

            return Json(response);
        }


        dynamic phone = null;
        var phonesList = dbContext.tbl_cliente_telefonos.Where(x => x.fk_id_cliente == client.id_clientes).OrderByDescending(x => x
        if (phonesList.Count > 0) phone = phonesList.FirstOrDefault(x => x.principal == true);
        if (phone is null) phone = phonesList.OrderBy(x => x.datacredito).FirstOrDefault();

        dynamic email = null;
        var emailList = dbContext.tbl_cliente_correos.Where(x => x.fk_id_cliente == client.id_clientes).OrderBy(x => x.datacredito)
        if (emailList.Count > 0) email = emailList.FirstOrDefault(x => x.datacredito == false);
        if (email is null) email = emailList.FirstOrDefault();

        var resp_cupo_disponible = await dbContext
            .Database
            .SqlQuery<AvailableCreditDto>("EXEC sp_cupo_disponible @numeroIdentificacion", new SqlParameter("@numeroIdentificacion"
            .FirstOrDefaultAsync();

        if (resp_cupo_disponible != null)
        {
            quotaId = resp_cupo_disponible.IdCupo;
            amountApproved = resp_cupo_disponible.MontoAprobado;
            amountAvailable = resp_cupo_disponible.MontoDisponible;
            balance = amountAvailable;
        }

        DatosFiltroDesembolsoDTO datosFiltroDesembolso = new DatosFiltroDesembolsoDTO();
        datosFiltroDesembolso.NumeroDocumento = client.numero_identificacion;
        datosFiltroDesembolso.NombreCliente = String.Join(" ", client.nombre, client.primer_apellido, client.segundo_apellido);
        result = desembolso.ValidacionInicialDesembolsar(datosFiltroDesembolso, permit, false);

        var sucursal = ConfigurationManager.AppSettings["Sucursal"];

        var idSucursal = int.Parse(sucursal);
        var infoSucursal = dbContext.tbl_oficinas.FirstOrDefault(x => x.id_oficina == idSucursal);
        var infoClient = desembolso.TraerInformacionCliente(datosFiltroDesembolso.NumeroDocumento, infoSucursal.codigo_interno, fal

        if (result.Status)...
        else...
    }
```

La operación consultar tendrá 2 objetivos, el primero es obtener la información del cliente que quiere realizar el crédito y validar si los datos recibidos son correctos y si tiene capacidad de cupo con respecto a la compra que quiere realizar. El segundo es crear y enviar un OTP al cliente para poder confirmar la compra.

```
dataObj = new
{
    status = "success",
    data = new
    {
        idcupo =  credito_vtex.fk_id_cupo,
        identificacion =  credito_vtex.numero_identificacion,
        diacorte =  desembolsoDto.DiaCorte,
        totalcupo =  amountApproved?.ToString("N0").Replace(",", "").Replace(".", ""),
        saldo =  balance?.ToString("N0").Replace(",", "").Replace(".", ""),
        verificacorreo =  (bool)credito_vtex.verificacorreo ? "Si" : "No",
        verificacelular =  (bool)credito_vtex.verificacelular ? "Si" : "No",
        verificacapacidad =  (bool)credito_vtex.verificacapacidad ? "Si" : "No",
        capacidad =  credito_vtex.capacidad?.ToString("N0").Replace(",", "").Replace(".", ""),
        token =  credito_vtex.otp,
        id_vtex =  credito_vtex.id_creditos_vtex,
        comentario = (bool)credito_vtex.verificacapacidad ? "Tiene capacidad" : "No tiene capacidad"
    },
    code = 200,
    mensaje = "Registros encontrados!"
};
```

Cumplido los dos objetivos se crea un objeto que tendrá como información el idCupo, el número de identificación, su día de corte, el saldo disponible y el total del cupo, las validaciones respectivas para permitir o no el desembolso y el id del crédito vtex que se crea en el proceso. Todo el objeto que contiene la información será encriptado antes de ser enviado como respuesta de la petición.

- **Operación Compensar**

Para este proceso se establece el valor “compensar” que tendrá la función de generar el pagare. Esta operación consiste en validar el código vtex recibido para luego generar el crédito en el sistema credi10.

Para esto se necesita recibir un objeto JSON en el cuerpo de la solicitud POST como el que se observa a continuación:

```
{
    "datos":"X4ntP6+ie0vX4WR+ljyUDvv+LJDCJO49RpE3t726Oav6FsHBsuZs9
        +cmPNCVDwPw5hNvCWSbidcsiZ4O1RTj4G025tLUQ6RV2DOkEaza6vI6WAaAYD9MbtLA12tjNon+ir2Rd\/6fubsaOJ6vM6U
        +9ehRghU\/g\/6+HSztGnjCR5k2glO\/QDZJllZNF3ttA5RrK\/
        mmTMBvXVbW7PQ7vBhKexRmyN1aWIQ57rYPHMbHAtjfIRxqcr88dYG0XOP191EFqiFwuSK7VgLTPZ9RcmF25eL7SdZNL
        +AiioKko5g1CcweR6ua5zvefDmJR4jX6kS9RlBY6RqPhi82Ve72flIiHTOtl5ThbAyChSlVJdeajz0="
}
```

Donde el contenido del atributo “datos” corresponde a una encriptación del siguiente objeto

```
{
“operacion”: “compensar”,
“idLiquida”: “37”,
“idcupo”: “456786”,
“monto”: “50000”,
“token”: “325689”,
“codsucursal”: “995”,
“plazo”:”1”,
“token_vtex”: “456895”,
“usuario”: “xxx”,
“password”: “xxx”
}
```

El objeto json recibido en la petición se desencripta con claves específicas y se inicia con el proceso validación del usuario y password suministrados, si no son correctas las

credenciales el sistema retornará “No Autorizado”, en caso de ser correctas, se procederá con la validación del OTP y posterior creación del crédito.

```
else if (dataCheckoutDto.Operacion == "compensar")
{
    var sucursal = ConfigurationManager.AppSettings["Sucursal"];

    using (var dbContext = new BaseContext())
    {
        long id_creditovtex = long.Parse(dataCheckoutDto.IdLiquida);
        var credito_vtex = dbContext.tbl_creditos_vtex.FirstOrDefault(x => x.id_creditos_vtex == id_creditovtex);
        if (credito_vtex != null)
        {

            //int id_sucursal = int.Parse(sucursal);
            var infoSucursal = dbContext.tbl_oficinas.FirstOrDefault(x => x.codigo_interno == dataCheckoutDto.CodSucursal);
            var infoClient = desembolso.TraerInformacionCliente(credito_vtex.numero_identificacion, infoSucursal.codigo_interno, false);
            if (infoClient.Status)
            {
                DatosDesembolsoDTO datosDesembolsoDTO = JsonConvert.DeserializeObject<DatosDesembolsoDTO>(infoClient.Data);

                List<ProgramacionCuotasDTO> cuotas = null;
                FormularioDesembolsoDTO formularioDesembolsoDTO = new FormularioDesembolsoDTO();
                formularioDesembolsoDTO.FechaExpedicionIngresada = datosDesembolsoDTO.FechaExpedicionDocumento;
                formularioDesembolsoDTO.CedulaAsesor = "";
                formularioDesembolsoDTO.OtpDigitado = int.Parse(dataCheckoutDto.Token);
                formularioDesembolsoDTO.NoCuotas = credito_vtex.cuotas ?? 1;
                formularioDesembolsoDTO.ValorDeCompra = dataCheckoutDto.Monto;

                UserDTO userDTO = new UserDTO();
                PermitDTO permitDTO = new PermitDTO();
                permitDTO.Crear = true;

                result = maestrosCreditosService.CrearCredito(datosDesembolsoDTO, cuotas, formularioDesembolsoDTO, userDTO, permitDTO, Contrc

                if (result.Completed)
                {
                    credito_vtex.token_vtex = dataCheckoutDto.Token_Vtex;
                    dbContext.SaveChanges();
                    if (result.Status)
                    {
                        Preaprobado.AceptacionHabeasDataYPagarePreaprobadoDesembolso(datosDesembolsoDTO.NumeroCedula, ControllerContext, usua

                    }
                    dataObj = new
                    {
                        status = "success",
                        data = new
                        {
                            pagare = result.Code
                        },
                        code = 200,
                        mensaje = "Valor discriminado y pagaré enviado!"
                    };
                }
                else
                {
                    dataObj = new
                    {
                        status = "error",
                        data = new List<string>(),
                        code = 405,
                        mensaje = result.Message
                    };
                }
            }
            else...
        }
        else...
    }
```

La operación compensar tendrá 2 objetivos que serán ejecutados en el método llamado *CrearCredito()*, estos objetivos son la creación del crédito y notificación del desembolso realizado por correo electrónico y SMS.

Cuando el proceso termine de ejecutarse se construirá un objeto anónimo con la información del resultado de la operación que será retornada como respuesta. Esta respuesta no estará encriptada.

Otros procesos

A. Validar el usuario



```
public bool ValidateUser(string username, string password)
{
    var pass = "";

    try
    {
        using (var dbContext = new BaseContext())
        {
            var result = dbContext.tbl_usuarios.FirstOrDefault(u => u.estado && u.login.ToUpper() == username.ToUpper());
            if (result != null)
            {
                if (result.cambio_clave != null) pass = SecurityUtil.Sha256(password);
                else pass = SecurityUtil.Md5(password);

                return result.password == pass;

            }
            return false;
        }
    }
    catch(Exception ex)
    {
        return false;
    }
}
```

Método que valida si el usuario y la contraseña enviada en la petición después de ser descifrada están registrados como usuarios en el sistema para poder acceder a la información en credi10.

B. Encriptar

```
public string Encrypt(string plainText)
{
    if (!string.IsNullOrEmpty(plainText))
    {
        try
        {
            using (Aes aesAlgorithm = Aes.Create())
            {
                string password = GetSha256Hash(Encoding.UTF8.GetBytes(SecretKey));
                byte[] passwordBytes = Encoding.UTF8.GetBytes(password);
                byte[] newPasswordBytes = new byte[32];
                Array.Copy(passwordBytes, 0, newPasswordBytes, 0, 32);
                aesAlgorithm.Key = newPasswordBytes;

                string IV = GetSha256Hash(Encoding.UTF8.GetBytes(SecretIV));
                byte[] ivBytes = Encoding.UTF8.GetBytes(IV);
                byte[] newIVBytes = new byte[16];
                Array.Copy(ivBytes, 0, newIVBytes, 0, 16);
                aesAlgorithm.IV = newIVBytes;

                // Create encryptor object
                ICryptoTransform encryptor = aesAlgorithm.CreateEncryptor();

                byte[] encryptedData;

                //Encryption will be done in a memory stream through a CryptoStream object
                using (MemoryStream ms = new MemoryStream())
                {
                    using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
                    {
                        using (StreamWriter sw = new StreamWriter(cs))
                        {
                            sw.Write(plainText);
                        }
                        encryptedData = ms.ToArray();
                    }
                }

                return Convert.ToBase64String(encryptedData);
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine("Error " + ex.Message);
            return plainText;
        }
    }
    return plainText;
}
```

(variable local) Aes aesAlgorithm

Método usado para encriptar información con AES (The Advanced Encryption Standard).

C. Desencriptar

```
public string Decrypt(string encryptedData)
{
    if (!string.IsNullOrEmpty(encryptedData))
    {
        try
        {
            using (Aes aesAlgorithm = Aes.Create())
            {
                string password = GetSha256Hash(Encoding.UTF8.GetBytes(SecretKey));
                byte[] passwordBytes = Encoding.UTF8.GetBytes(password);
                byte[] newPasswordBytes = new byte[32];
                Array.Copy(passwordBytes, 0, newPasswordBytes, 0, 32);
                aesAlgorithm.Key = newPasswordBytes;

                string IV = GetSha256Hash(Encoding.UTF8.GetBytes(SecretIV));
                byte[] ivBytes = Encoding.UTF8.GetBytes(IV);
                byte[] newIVBytes = new byte[16];
                Array.Copy(ivBytes, 0, newIVBytes, 0, 16);
                aesAlgorithm.IV = newIVBytes;

                // Create decryptor object
                ICryptoTransform decryptor = aesAlgorithm.CreateDecryptor();

                byte[] cipher = Convert.FromBase64String(encryptedData);

                //Decryption will be done in a memory stream through a CryptoStream object
                using (MemoryStream ms = new MemoryStream(cipher))
                {
                    using (CryptoStream cs = new CryptoStream(ms, decryptor, CryptoStreamMode.Read))
                    {
                        using (StreamReader sr = new StreamReader(cs))
                        {
                            return sr.ReadToEnd();
                        }
                    }
                }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine("Error " + ex.Message);
            return encryptedData;
        }
    }
    return encryptedData;
}
```

Método usado para desencriptar una cadena de string que este encriptado con AES (The Advanced Encryption Standard)

D. Crear Crédito

Método que crea el desembolso con respecto a los datos enviados desde el botón ecommerce. Explicado en el documento GenerarCredito.docx.

1.3.4 Documentación de Arquitectura

Lógica completa para crear un desembolso GenerarCredito.docx

Demas procesos del API de servicios:

- B2C.docx

- LinkDePagos.docx

1.3.5 Log

En este proceso no es generado ningún log de auditoría.

1.3.6 Lista de Programas, archivos, paquetes

- **sp_cupo_disponible:**
  Procedimiento almacenado que retorna el valor del cupo aprobado y el monto disponible que tiene en credi10 de un cliente especifico.

1.3.7 Lista de modificaciones hacia la base de datos

Script 1. Creación de tablas: Para crear el esquema completo de la base de datos de la aplicación.

Creacion_BD_C10_Credi10Net.sql

1.4 Observaciones y/o Comentarios

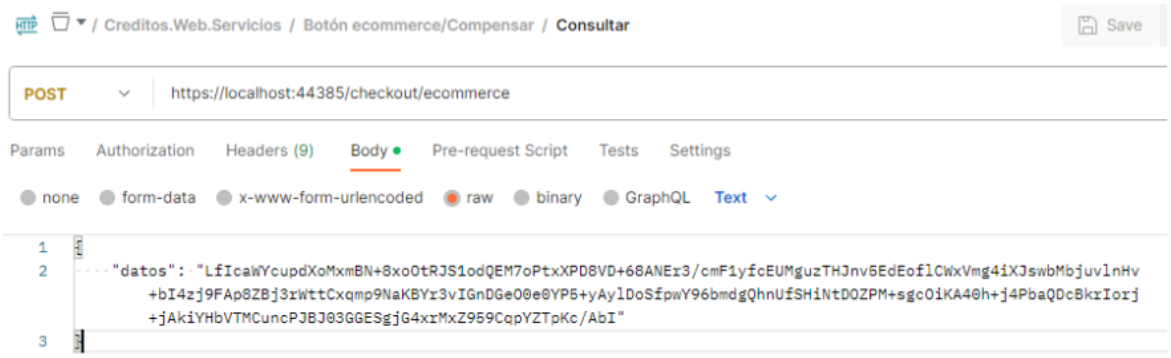
En el archivo coleccionPostman.json se encontrará la colección que podrá ser exportada a postman con todas las consultas que podrán ser realizadas al API de servicios.

2. CASOS DE PRUEBAS UNITARIAS

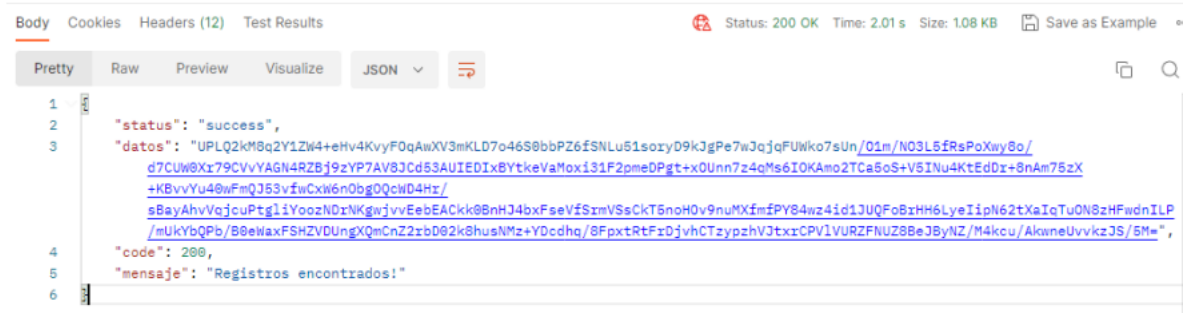
A continuación, se demuestra las pruebas funcionales hechas en el ambiente de pruebas.

Para llevar a cabo las pruebas de consulta de botón ecommerce, se ejecuta el request en Postman y se obtiene la siguiente respuesta satisfactoria:

- Envió de consulta de botón ecommerce

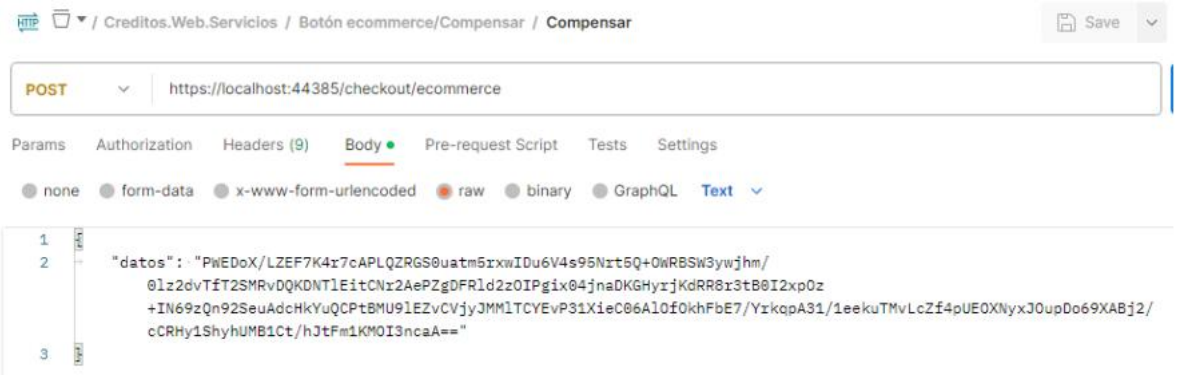


- Respuesta de consulta de boton ecommerce

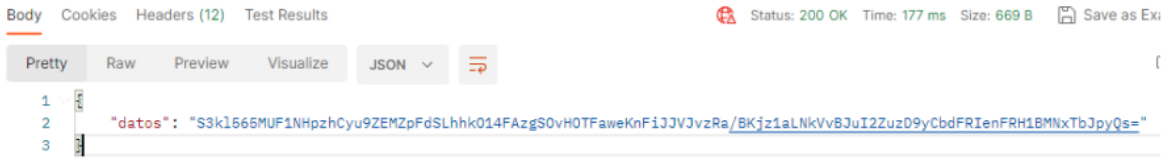


Para llevar a cabo las pruebas de registro de pagaré de botón ecommerce, se ejecuta el request en Postman y se obtiene la siguiente respuesta satisfactoria:

- Envió de información para registro de pagare de botón ecommerce



- Respuesta de registro de pagare de botón ecommerce



ID	CASO DE PRUEBA	RESULTADO ESPERADO	APROBADO
1	Ejecución de API consulta botón ecommerce	Información de consulta botón ecommerce con encriptación	SI
2	Ejecución de API generación botón ecommerce	Confirmación de pagare generado con encriptación	SI